GUIDE VISUEL · C# POUR DÉBUTANTS

Construire un backend ASP.NET Core

Les imports, l'architecture en couches et l'injection de dépendances, expliqués pas à pas avec le code de ton projet Portfolio.

C# 14 · .NET 10 | Controllers | Services | Repositories | EF Core | PostgreSQL | xUnit

.NET

PROGRAMME

Ce que tu vas comprendre

1. **Les bases C# utiles**
   class, interface, record, async/await
2. **Les imports**
   namespace, using, paquets NuGet
3. **L'architecture en couches**
   le trajet d'une requête
4. **Construire chaque couche**
   Entity → Repository → Service → Controller
5. **L'injection de dépendances**
   enregistrer, recevoir, durées de vie
6. **Program.cs, config et base**
   pipeline, appsettings, migrations
7. **Tester son code**
   xUnit + faux objets
8. **Recette et pièges**
   ajouter une fonctionnalité, erreurs fréquentes

> **Astuce :** garde le projet ouvert à côté. Chaque exemple vient d'un vrai fichier de `backend/src/Portfolio.Api/`.

01

Les bases C# utiles

Le minimum de vocabulaire pour lire un backend sans être perdu.

Le vocabulaire en une page

class

Un « moule » qui contient des données et du code.

```
public class Project
{
    public int Id { get; set; }
}
```

interface

Un **contrat** : la liste de ce qu'on sait faire, sans le code.

```
public interface IProjectService
{
    Task<ProjectDto?>
        GetByIdAsync(int id);
}
```

record

Une classe courte pour **transporter** des données.

```
public record MeResponse(
    string Email);
```

{ get; set; }

Une **propriété** : une donnée qu'on peut lire (get) et modifier (set).

? nullable

`string?` = peut valoir `null` (vide).
`string` = jamais null.

required

Obligatoire à la création de l'objet. Le compilateur refuse s'il manque.

async / await : attendre sans bloquer

Services/ProjectService.cs

```
public async Task<ProjectDto?> GetByIdAsync(int id)
{
    // On "attend" la base de données…
    var project = await repository.GetByIdAsync(id);
    // …puis on continue avec le résultat
    return project?.ToDto();
}
```

Au restaurant : tu commandes, et pendant que ça cuit, le serveur sert d'autres tables. Le serveur web fait pareil pendant qu'il attend la base.

Les 4 règles

- Une méthode qui attend quelque chose est marquée `async`.
- Elle renvoie `Task` (rien) ou `Task<T>` (une valeur de type T).
- On met `await` devant chaque appel asynchrone.
- Par convention, son nom finit par `Async`.

Le constructeur principal

Tu le verras partout dans le projet. Ces deux versions font **exactement** la même chose.

Forme classique

```
public class ProjectService : IProjectService
{
    private readonly IProjectRepository _repository;

    public ProjectService(IProjectRepository repository)
    {
        _repository = repository;
    }
}
```

Forme courte (C# 12+)

```
public class ProjectService(IProjectRepository repositor
    : IProjectService
{
    // "repository" est utilisable
    // dans toutes les méthodes
}
```

> **À retenir :** les paramètres entre parenthèses après le nom de la classe = ce dont la classe **a besoin** pour fonctionner. C'est la porte d'entrée de l'injection de dépendances (chapitre 5).

`: IProjectService` signifie « cette classe respecte le contrat IProjectService ».

02

Les imports

namespace, using, paquets NuGet : qui fait quoi, et comment réparer « type introuvable ».

2 IMPORTS

Un namespace, c'est une adresse

Services/ProjectService.cs

```
namespace Portfolio.Api.Services;

public class ProjectService { … }
```

Le nom complet de la classe devient :

`Portfolio.Api.Services.ProjectService`

> **Comme une adresse postale :** plusieurs classes peuvent s'appeler pareil si elles n'habitent pas à la même adresse.

Convention : namespace = dossier

```
Portfolio.Api/
├─ Services/
│    → Portfolio.Api.Services
├─ Repositories/
│    → Portfolio.Api.Repositories
└─ DTOs/
     → Portfolio.Api.DTOs
```

2 IMPORTS

`using` = « j'utilise ce qui habite à cette adresse »

Controllers/ProjectsController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Portfolio.Api.DTOs;
using Portfolio.Api.Services;

namespace Portfolio.Api.Controllers;

[ApiController]
public class ProjectsController(IProjectService projectService)
    : ControllerBase
{
    [HttpGet("{id:int}")]
    public async Task<ActionResult<ProjectDto>> GetById(int id) …
}
```

Qui apporte quoi ?

`…Authorization`	`[Authorize]`
`…Mvc`	`ControllerBase`, `[HttpGet]`, `ActionResult`
`…DTOs`	`ProjectDto`
`…Services`	`IProjectService`

Sans le bon `using`, le compilateur ne trouve pas le type.

2 IMPORTS

Paquet, using, référence : trois choses différentes

📦

Paquet NuGet

Télécharge une bibliothèque dans le projet.

```
dotnet add package NSubstitute
```

Une fois par projet. Ajouté dans le `.csproj`.

📄

using

Rend un namespace visible **dans un fichier**.

```
using NSubstitute;
```

En haut de chaque fichier qui en a besoin.

🔗

Référence de projet

Un projet voit le code d'un autre projet.

```
dotnet add tests/… reference src/…
```

Ex. : les tests voient l'API.

Usings implicites : grâce à `<ImplicitUsings>enable</ImplicitUsings>`, `System`, `System.Linq`, `System.Threading.Tasks`… sont déjà importés partout.

Erreur CS0246 « type introuvable » → place le curseur sur le mot rouge et fais **Ctrl + .** : VS Code propose le bon `using`. Rien ne s'affiche ? Il manque sans doute le paquet.

03

L'architecture en couches

Chaque couche a un seul métier. Une requête les traverse dans l'ordre, puis la réponse fait le chemin inverse.

3 ARCHITECTURE

Le trajet d'une requête

`GET /api/projects/1` : « donne-moi le projet n° 1 »



Retour : Entity

Le repository renvoie un objet `Project`, le reflet exact de la table.

Retour : DTO

Le service le convertit en `ProjectDto` : seulement ce qu'on veut montrer.

Retour : JSON

Le controller renvoie `200 OK` + le DTO en JSON, ou `404` si rien trouvé.

Pourquoi séparer ? L'analogie du restaurant



Le serveur = Controller

Prend la commande et rapporte l'assiette. Il **ne cuisine pas**.



Le cuisinier = Service

Suit la recette (les règles métier). Il ne parle **pas aux clients**.



Le magasinier = Repository

Sait où sont rangés les ingrédients dans la réserve (la base de données).

Bénéfice 1 : on peut changer de réserve (PostgreSQL → autre chose) sans toucher au cuisinier.

Bénéfice 2 : on peut tester le cuisinier avec un faux magasinier (chapitre 7).

Et le menu ? C'est le **DTO** : ce que le client voit, pas la recette interne.

3 ARCHITECTURE

Où se trouve chaque chose

```
src/Portfolio.Api/
├─ Controllers/      HTTP : routes, codes 200/404…
├─ Services/         logique métier (I… + classe)
├─ Repositories/     accès aux données (I… + classe)
├─ DTOs/             ce qui entre / sort de l'API
├─ Entities/         les tables de la base
├─ Mappings/         Entity ⇄ DTO
├─ Data/             DbContext + migrations
├─ Extensions/       enregistrement de la DI
├─ appsettings.json  configuration
└─ Program.cs        point de départ
```

Règle d'or

Une couche ne parle qu'à **celle juste en dessous**.

Controller → Service → Repository. Jamais Controller → Repository directement.

I devant le nom

`IProjectService` = l'interface (le contrat).
`ProjectService` = la classe qui le réalise.

04

Construire chaque couche

On part de la base de données et on remonte jusqu'au controller.

Entity : une classe = une table

Entities/Project.cs

```
namespace Portfolio.Api.Entities;

public class Project
{
    // "Id" = clé primaire automatique
    public int Id { get; set; }
    public required string Title { get; set; }
    public required string Description { get; set; }
    // "?" = la colonne peut être vide
    public string? RepoUrl { get; set; }
    public DateTime CreatedAt { get; set; }
        = DateTime.UtcNow;
}
```

Data/AppDbContext.cs

```
public class AppDbContext(
    DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    // Chaque DbSet = une table
    public DbSet<Project> Projects => Set<Project>();
    public DbSet<AdminUser> AdminUsers
        => Set<AdminUser>();

    protected override void OnModelCreating(
        ModelBuilder mb)
    {
        mb.Entity<Project>()
          .Property(p => p.Title).HasMaxLength(200);
    }
}
```

EF Core (Entity Framework Core) lit ces classes et crée les tables. Une propriété nommée `Id` devient automatiquement la clé primaire, auto-incrémentée.

Repository : le seul qui parle à la base

1. Le contrat

Repositories/IProjectRepository.cs

```
public interface IProjectRepository
{
    Task<List<Project>> GetAllAsync();
    Task<Project?> GetByIdAsync(int id);
    Task AddAsync(Project project);
    void Remove(Project project);
    Task SaveChangesAsync();
}
```

2. La réalisation avec EF Core

Repositories/ProjectRepository.cs

```
public class ProjectRepository(AppDbContext db)
    : IProjectRepository
{
    public Task<Project?> GetByIdAsync(int id) =>
        db.Projects.FirstOrDefaultAsync(p => p.Id == id);

    public async Task AddAsync(Project project) =>
        await db.Projects.AddAsync(project);

    public Task SaveChangesAsync() => db.SaveChangesAsync();
    // …
}
```

Important : `AddAsync` et `Remove` préparent les changements. Rien n'est écrit en base avant `SaveChangesAsync()`.

4 COUCHES · SERVICE

Service : la logique métier

Services/ProjectService.cs

```
public class ProjectService(IProjectRepository repository)
    : IProjectService
{
    public async Task<ProjectDto?> UpdateAsync(
        int id, ProjectRequest request)
    {
        var project = await repository.GetByIdAsync(id);
        // Le projet n'existe pas ?
        if (project is null) return null;

        // DTO → Entity, puis on enregistre
        request.ApplyTo(project);
        await repository.SaveChangesAsync();

        // Entity → DTO
        return project.ToDto();
    }
}
```

Ce que fait un service

- Vérifie les règles (« ce projet existe-t-il ? »).
- Convertit DTO ⇄ Entity.
- Orchestre les appels au repository.

Ce qu'il ne fait pas

- Pas de HTTP (ni 404, ni cookies).
- Pas d'EF Core direct (pas de `db.Projects`).

4 COUCHES · DTO

DTO : ce qui entre et sort de l'API

DTOs/ProjectDtos.cs

```
// Ce qu'on RENVOIE au client
public record ProjectDto(
    int Id,
    string Title,
    string Description,
    string? RepoUrl,
    string? LiveUrl,
    DateTime CreatedAt);

// Ce qu'on REÇOIT du client (avec validation)
public record ProjectRequest(
    [Required, StringLength(200)]  string Title,
    [Required, StringLength(4000)] string Description,
    [Url, StringLength(500)] string? RepoUrl,
    [Url, StringLength(500)] string? LiveUrl);
```

Pourquoi ne pas renvoyer l'Entity ?

`AdminUser` contient `PasswordHash`. Renvoyer l'entité, c'est risquer de le divulguer. Le DTO choisit **exactement** les champs exposés.

Validation automatique

Les attributs `[Required]`, `[Url]`… sont vérifiés **avant** d'entrer dans le controller. Invalide → `400 Bad Request` sans écrire une ligne.

Controller : traduire en HTTP

Controllers/ProjectsController.cs

```
[ApiController]
[Route("api/projects")]
public class ProjectsController(IProjectService projectService)
    : ControllerBase
{
    [HttpGet("{id:int}")]              // GET /api/projects/5
    public async Task<ActionResult<ProjectDto>> GetById(int id)
    {
        var project = await projectService.GetByIdAsync(id);
        if (project is null) return NotFound(); // 404
        return Ok(project);                     // 200 + JSON
    }

    [HttpPost]                         // POST /api/projects
    [Authorize(Policy = AuthSetup.AdminPolicy)] // admin seulement
    public async Task<ActionResult<ProjectDto>> Create(
        ProjectRequest request)
    {
        var project = await projectService.CreateAsync(request);
        return CreatedAtAction(nameof(GetById),
            new { id = project.Id }, project);  // 201
    }
}
```

Les attributs

`[ApiController]`	validation auto, erreurs JSON
`[Route]`	préfixe d'URL
`[HttpGet]` …	verbe HTTP + suite de l'URL
`[Authorize]`	il faut être connecté

Les réponses

Ok() 200 **NotFound()** 404
NoContent() 204 **Created…** 201

05

L'injection de dépendances

Le concept clé d'ASP.NET Core. Une fois compris, tout le reste devient logique.

5 INJECTION DE DÉPENDANCES

Le problème : tout fabriquer soi-même

Sans DI, pour utiliser le controller, il faudrait construire toute la chaîne à la main :

```
var options    = new DbContextOptionsBuilder<AppDbContext>()
                     .UseNpgsql("Host=localhost;…").Options;
var db         = new AppDbContext(options);
var repository = new ProjectRepository(db);
var service    = new ProjectService(repository);
var controller = new ProjectsController(service); // enfin !
```

😩 Répétitif

À refaire pour chaque requête et chaque controller.

🔒 Figé

Impossible de remplacer le repository par un faux pour les tests.

🧹 Fuites

Qui ferme la connexion à la base, et quand ?

La solution : demander, pas fabriquer

Une classe **déclare** ce dont elle a besoin dans son constructeur. ASP.NET Core le lui **fournit** automatiquement.

```
// "J'ai besoin d'un IProjectService."
public class ProjectsController(
    IProjectService projectService)
    : ControllerBase
{ … }
```

Le cuisinier ne construit pas son four. Il arrive en cuisine et le four est déjà là, installé par le restaurant.

Le conteneur de services

C'est le « majordome » d'ASP.NET Core. Il fonctionne en 2 temps :

1. **Au démarrage**, tu lui donnes un **catalogue** : « IProjectService → utilise ProjectService ».
2. **À chaque requête**, il crée les objets demandés et leurs propres dépendances, dans le bon ordre.

Il gère aussi leur durée de vie et les nettoie à la fin.

5 INJECTION DE DÉPENDANCES · ÉTAPE 1

Enregistrer : remplir le catalogue

Extensions/ServiceCollectionExtensions.cs

```
public static IServiceCollection AddRepositories(
    this IServiceCollection services) =>
    services
        .AddScoped<IProjectRepository, ProjectRepository>()
        .AddScoped<IAdminUserRepository, AdminUserRepository>();

public static IServiceCollection AddApplicationServices(
    this IServiceCollection services) =>
    services
        .AddSingleton<IPasswordHasher<AdminUser>,
                      PasswordHasher<AdminUser>>()
        .AddScoped<IProjectService, ProjectService>()
        .AddScoped<IAuthService, AuthService>();
```

Comment lire une ligne

`AddScoped<IProjectService, ProjectService>()`

« Quand quelqu'un demande un IProjectService, donne-lui un ProjectService. »

Puis dans Program.cs

```
builder.Services
    .AddDatabase(builder.Configuration)
    .AddRepositories()
    .AddApplicationServices();
```

5 INJECTION DE DÉPENDANCES · ÉTAPE 2

Recevoir : le conteneur remonte la chaîne

Requête `GET /api/projects/1` : le conteneur doit créer un `ProjectsController`…

🎮 `ProjectsController` a besoin de `IProjectService`

↳ catalogue : 🧠 crée `ProjectService`, qui a besoin de `IProjectRepository`

↳ catalogue : 🗄️ crée `ProjectRepository`, qui a besoin de `AppDbContext`

↳ catalogue : ⚙️ crée `AppDbContext` avec la chaîne de connexion de la config

Tu n'écris jamais `new ProjectService(…)`. Tu déclares le besoin, le conteneur fait le reste, puis libère tout à la fin de la requête.

Oubli d'enregistrement ? Erreur au démarrage : `Unable to resolve service for type 'IProjectService'`

Les 3 durées de vie



AddSingleton

Un seul objet pour toute la vie de l'application.

Comme la machine à café du bureau : partagée par tous.

Projet : `PasswordHasher` (aucun état, sans risque à partager).



AddScoped par défaut

Un objet par requête HTTP, partagé pendant cette requête.

Comme un plateau repas : un par client.

Projet : services, repositories, `AppDbContext`.



AddTransient

Un nouvel objet à chaque fois qu'on le demande.

Comme une serviette en papier : jetable.

Pour de petits outils légers, sans état.

Piège classique : un Singleton ne doit jamais recevoir un Scoped (ex. le DbContext). Il le garderait pour toujours. ASP.NET Core le détecte en développement : `Cannot consume scoped service … from singleton`.

06

Program.cs, config et base

Le point de départ de l'application, la configuration et l'évolution de la base de données.

6 PROGRAM.CS

Deux phases : construire, puis brancher

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// ① Le catalogue de services (DI)
builder.Services
    .AddDatabase(builder.Configuration)
    .AddRepositories()
    .AddApplicationServices()
    .AddAdminAuth(builder.Environment);
builder.Services.AddControllers();

var app = builder.Build();

// ② Le pipeline : chaque requête traverse ces étapes
app.UseHttpsRedirection();
app.UseAuthentication();   // qui es-tu ?
app.UseAuthorization();    // as-tu le droit ?
app.UseRateLimiter();
app.MapControllers();      // → le bon controller

app.Run();
```

① `builder.Services`

Avant `Build()` : on **déclare** ce qui existe. L'ordre importe peu.

② `app.Use…`

Après `Build()` : les **middlewares**, des portiques que chaque requête traverse. **L'ordre compte !**

Exemple : `UseAuthentication` doit venir avant `UseAuthorization`. On vérifie l'identité avant les droits.

6 CONFIGURATION

La configuration en couches

Chaque source peut écraser la précédente. La dernière gagne.

appsettings.json
valeurs communes

→

appsettings.Development.json
surcharges en dev

→

user-secrets
mots de passe en dev, hors du dépôt git

→

Variables d'env.
en production

appsettings.Development.json

```
{
  "ConnectionStrings": {
    "Default": "Host=localhost;Database=portfolio;…"
  },
  "Database": { "MigrateOnStartup": true }
}
```

```
// Lire une valeur (IConfiguration est injectable !)
config.GetConnectionString("Default");
config.GetValue<bool>("Database:MigrateOnStartup");
config["Admin:Email"];

// En variable d'env., ":" devient "__"
// Admin__Email=moi@site.fr
```

Ne commite jamais un mot de passe. En dev : `dotnet user-secrets set "Admin:Password" "…"`.

6 EF CORE

Faire évoluer la base : les migrations

Une migration = un fichier C# qui décrit un changement de schéma. Il est versionné avec le code.




```
# depuis backend/
dotnet ef migrations add AjoutCompetences --project src/Portfolio.Api --output-dir Data/Migrations
dotnet ef database update --project src/Portfolio.Api
```

Dans ce projet, l'étape 3 est automatique en dev : au démarrage, `DatabaseInitializer` applique les migrations manquantes.

07

Tester son code

Grâce à la DI et aux interfaces, on peut tester un service seul, en lui donnant un faux repository.

Un test unitaire avec un faux repository

tests/.../Unit/ProjectServiceTests.cs

```
public class ProjectServiceTests
{
    // Un FAUX repository : aucune base de données !
    private readonly IProjectRepository _repository =
        Substitute.For<IProjectRepository>();

    [Fact]
    public async Task UpdateAsync_returns_null_when_missing()
    {
        // Arrange : le faux répond "rien trouvé"
        _repository.GetByIdAsync(1).Returns((Project?)null);
        var service = new ProjectService(_repository);

        // Act
        var result = await service.UpdateAsync(1, Request);

        // Assert
        Assert.Null(result);
        await _repository.DidNotReceive().SaveChangesAsync();
    }
}
```

Arrange · Act · Assert

- **Arrange :** préparer la situation.
- **Act :** appeler la méthode testée.
- **Assert :** vérifier le résultat.

Deux types de tests

Unit/ : une classe seule, avec des faux objets. Très rapide.

Integration/ : toute l'API démarrée en mémoire, avec de vraies requêtes HTTP.

Lancer : `dotnet test`

08

Recette et pièges

La marche à suivre pour ajouter une fonctionnalité, et les erreurs que tout le monde rencontre.

8 RECETTE

Ajouter une fonctionnalité « Compétences »

1. **Entity** `Entities/Skill.cs` (Id, Name, Level)
2. **DbSet** dans `AppDbContext` : `Skills`
3. **Migration** `dotnet ef migrations add AddSkills`
4. **DTOs** `SkillDto` + `SkillRequest` (avec validation)
5. **Mapping** `ToDto()`, `ToEntity()`
6. **Repository** `ISkillRepository` + `SkillRepository`
7. **Service** `ISkillService` + `SkillService`
8. **Enregistrer** les deux dans `ServiceCollectionExtensions`
9. **Controller** `SkillsController` + `[Route("api/skills")]`
10. **Tests** unitaires du service + un test d'intégration

> **Astuce :** copie les fichiers de Project, puis renomme. La structure est toujours la même ; seul le métier change.

8 PIÈGES

Les erreurs les plus fréquentes

SYMPTÔME	CAUSE	SOLUTION
`Unable to resolve service for type 'IXxx'`	Service non enregistré dans la DI	Ajouter `.AddScoped<IXxx, Xxx>()`
`CS0246` type ou namespace introuvable	`using` ou paquet manquant	**Ctrl + .** sur le mot, ou `dotnet add package`
Les données ne sont pas enregistrées	Oubli de `SaveChangesAsync()`	L'appeler après Add / Remove / modification
Avertissement `CS4014`, résultat bizarre	Oubli de `await`	Mettre `await` devant chaque appel `…Async`
`Cannot consume scoped service … from singleton`	Mauvaise durée de vie	Passer le service en `AddScoped`
Toujours `401` alors que je suis connecté	Ordre des middlewares	`UseAuthentication()` avant `UseAuthorization()`

8 MÉMO

Les commandes à garder sous la main

Au quotidien (depuis backend/)

```
docker compose up -d                                          # démarrer PostgreSQL
dotnet build                                                  # compiler
dotnet run   --project src/Portfolio.Api --launch-profile http # lancer l'API
dotnet watch --project src/Portfolio.Api                      # lancer + relance auto
dotnet test                                                   # lancer les tests
```

Plus rarement

```
dotnet add src/Portfolio.Api package NomDuPaquet                                  # ajouter une bibliothèque
dotnet ef migrations add NomMigration --project src/Portfolio.Api --output-dir Data/Migrations
dotnet ef database update --project src/Portfolio.Api                             # appliquer les migrations
dotnet user-secrets set "Cle:SousCle" "valeur" --project src/Portfolio.Api        # secret de dev
dotnet tool restore                                                               # après un git clone
```

📘 Doc officielle

learn.microsoft.com/aspnet/core

🧩 DI en détail

Cherche « dependency injection ASP.NET Core »

🗃️ EF Core

learn.microsoft.com/ef/core

RÉCAP

Ce qu'il faut retenir

using importe un namespace (une adresse). Le paquet NuGet, lui, apporte la bibliothèque.

Couches : Controller (HTTP) → Service (métier) → Repository (données).

DTO pour l'extérieur, **Entity** pour la base.

DI : on **demande** dans le constructeur, on **enregistre** dans le catalogue.

Scoped par défaut pour un backend web.

Interfaces = on peut tester avec des faux objets.

Exercice : ajoute la fonctionnalité « Compétences » en suivant la recette. 🚀